HOMELAB MONITORING & CONTROL FABRIC

solariNet

Developer & Architecture Reference — the implemented and planned system, end to end.

DOCUMENT
SolariNet_Developer_Reference

REVISION
1.1 · 2026-10-21

SOURCE BRANCH
integrate/stability-20261020

SOURCE COMMIT
1a71e8b

REPOSITORY
github.com/soren42/Solarinet

INTERNAL

ABOUT THIS DOCUMENT

Document control

This reference describes SolariNet — a self-hosted network monitoring, discovery, and control fabric written in standard C with a PHP dashboard — as it is actually built on the `akoria` homelab, alongside the architecture it is planned to become under the current stabilization program.

> **CURRENT CANDIDATE STATUS · 2026-10-21**
>
> This document combines historical architecture and inventory material with newer candidate and live-state notes. The current local integration candidate is branch `integrate/stability-20261020` at `1a71e8b`, 18 commits ahead of `main`; at baseline capture it had 131 tracked paths modified and 47 untracked paths. It has not been published or deployed. The highest release blockers from the current audit are the PHP/C privilege boundary, lease starvation during long scans, and HA lease/standby behavior. A losing-claim epoch defect is corrected in the candidate and passed the disposable live-DB suite, but remains unreviewed and unrehearsed. Certificate identity enforcement and full manual coverage also remain incomplete. See the stabilization baseline for test and live-host evidence. The §5 host inventory is a dated snapshot and was not refreshed by this revision.

How this was assembled

The contents were reconstructed from ground truth, not memory: the committed source tree on the branch and commit identified above; rendered configuration from the primary host `xenon`; the DNS source-of-truth `netdb/akoria-hosts.yml`; and an authorized live discovery pass across the homelab (host-liveness sweep plus read-only SSH fact-gathering, captured 2026-09-17). Later candidate and live observations are dated in the current-state callout and in the stabilization baseline. Network discovery on this infrastructure is routine, authorized administration of the operator's own network.

Reading the provenance flags

Every non-trivial claim is marked so a reader can tell measured fact from reasoning:

`OBSERVED` Read directly from source code, live host state, or a discovery scan during assembly.

`INFERRED` Deduced from surrounding evidence; not directly confirmed. Treat as a lead, not a guarantee.

Capability maturity is shown with the same labels the dashboard uses (planned track I0):

`PROTOTYPE` `LAB` `PRODUCTION` `AUTHORITATIVE`

A feature marked `LAB` works but is not trusted for unattended production use; several such capabilities are explicitly frozen (see §10).

> **SCOPE & HONESTY NOTE**
>
> SolariNet is under a declared **feature freeze**. Its own architecture review found that feature breadth had outrun enforcement and durability. This document therefore separates what is implemented and load-bearing from what exists in code but is not yet trustworthy — the distinction is the point, not a footnote.

Conventions

- Hostnames, IP/MAC addresses, ports, node IDs, file paths, verbs, and code are set in `IBM Plex Mono` — the data face.

SOLARINET

- Hosts are named for periodic-table elements and compounds (an RFC 2100 convention); the name carries no functional meaning. Function is expressed as a DNS `CNAME`.
- The control-plane binaries are `solariClient`, `solariMonitor`, `solariServer`; the wire protocol is `SCP` (SolariNet Control Protocol).

CONTENTS

Table of contents

01 System overview . . . 5

02 Control-plane architecture . . . 8

2.1 Three-tier topology & SCP transport . . . 8

2.2 solariServer internals & the solariCtl bridge . . . 9

03 Subsystem container views . . . 12

04 Network topology & data flows . . . 14

05 System inventory . . . 16

5.1 Subnets . . . 16

5.2 Host inventory — infrastructure & services . . . 17

5.3 Host inventory — endpoints, IoT & peripherals . . . 19

06 Component reference . . . 22

07 Data model & System of Record . . . 24

08 Identity, authorization & trust boundaries . . . 26

09 Developer workflow & build . . . 28

10 Operations & deployment . . . 30

11 Planned architecture & the stabilization program . . . 32

12 Appendices . . . 36

Page numbers are resolved at render time from the paginated document.

SECTION 01

System overview

SolariNet is a self-hosted fabric that watches a homelab and acts on it: edge agents report host health, a monitor tier probes reachability, and a central server persists state, raises alerts, drives discovery, and brokers operator commands — all in standard C, fronted by a build-step-free PHP dashboard.

What it is

The system is a clean **three-tier control plane** — `solariClient` (per-host agent), `solariMonitor` (on-the-wire prober), and `solariServer` (ingest, storage, alerting, control) — speaking a single custom binary protocol, SCP, over mutually-authenticated TLS. Around that core sit a MariaDB datastore, a RabbitMQ notification bus, a DNS/AD System of Record with change-data-capture, and an identity stack (Keycloak federated to Samba Active Directory). The operator sees all of it through one dashboard served by Apache and PHP-FPM.

The codebase holds to two hard house rules that shape every decision below: the application is **standard C with associated Makefiles**, and the dashboard is **PHP with no build step** — React and Babel are vendored and JSX is transpiled in the browser at load. There is no Node.js in the runtime, no bundler, and no server-side JavaScript anywhere. OBSERVED (no `package.json`, `composer.json`, or lockfile exists in the tree).

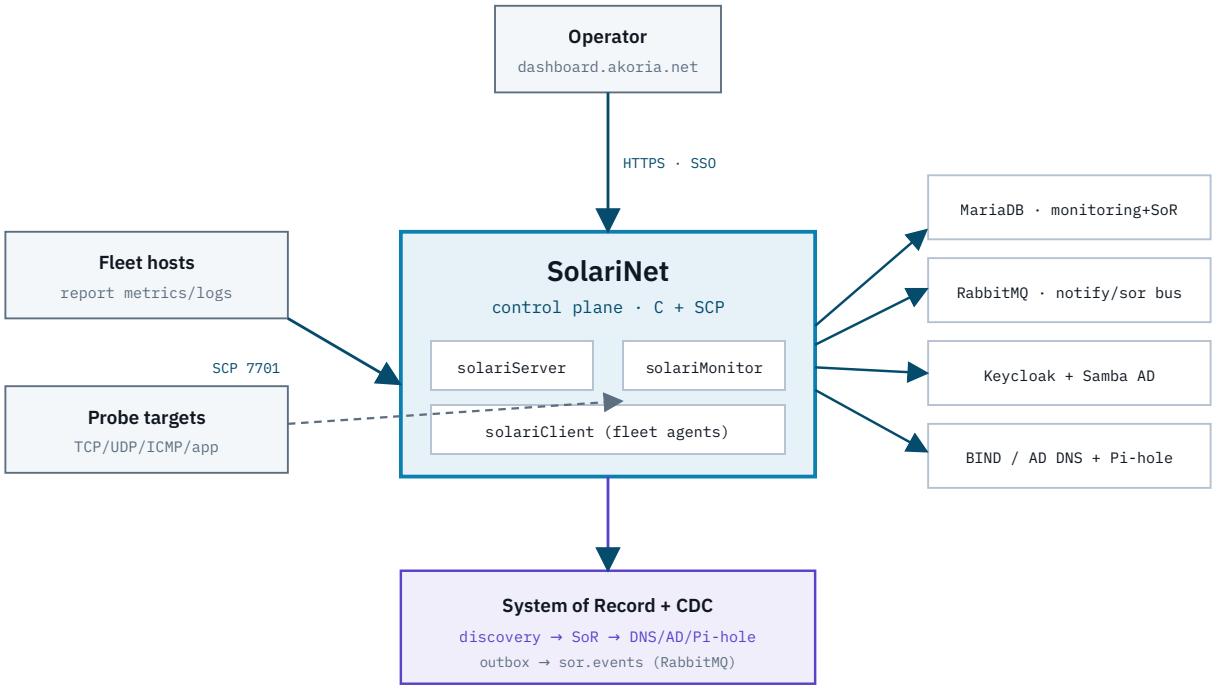


Figure 1.1 — System context. SolariNet's C control plane at center; operators, fleet, and probe targets around it; datastore, bus, identity, and DNS as backing services. Provenance: OBSERVED from source + live config.

The honest caveat: a fleet-of-one in practice

READ THIS BEFORE TRUSTING THE TOPOLOGY

The architecture is a genuine three-tier split, and the monitor tier's rendezvous-hash ownership is implemented in code. **In production, however, nearly every node runs the client role only**; the monitor tier and its k-of-n redundancy are not yet load-bearing. Treat multi-monitor resilience as designed-and-coded, not proven. Section 11 tracks the work (F-19/F-20) that is meant to make the claim true.

Technology stack

CORE TECHNOLOGY CHOICES — FROM DOCS/ARCHITECTURE.MD

LAYER	CHOICE	WHY (PER RATIONALE DOC)	FLAG
Control-plane language	ISO C99	Portable, dependency-light, functions-only; opaque-handle pattern, no OOP emulation.	OBS
Wire transport	nng + mbedTLS	Scalable-protocol sockets over mutually-authenticated TLS (tls+tcp://).	OBS
Wire protocol	SCP	Length-prefixed 32-byte big-endian header + TLV payload + CRC-32; golden-frame fixtures immutable per version.	OBS
Edge spool	SQLite	Durable client-side store-and-forward when the server is unreachable.	OBS

LAYER	CHOICE	WHY (PER RATIONALE DOC)	FLAG
Datastore	`MariaDB / InnoDB`	Monitoring DB + System of Record; all timestamps stored UTC.	OBS
Message bus	`RabbitMQ`	Durable topic exchanges for notifications and SoR change events; Mosquitto for device MQTT.	OBS
Dashboard	`PHP 8.x + Apache`	No framework, no Composer, no build step; React/Babel vendored, JSX transpiled in-browser.	OBS
Identity	`Keycloak (OIDC)`	Confidential-client auth-code flow, federated to Samba AD over LDAPS.	OBS
Support daemons	`Python (deploy/only)`	Discovery, notify, sorsync, unifi, etc. — never inside the C/PHP tiers.	OBS

Two domains, one fabric

DNS is split by purpose. `akoria.net` is the general forward zone, authoritative on `xenon` (BIND primary); the forward zone has secondaries on `radium` and `chlorine`, while generated reverse zones use `xenon` and `chlorine` because radium does not serve those zones. The ZimaBlade still labeled `steel` is offline and is not currently advertised as a nameserver; it is planned for a tertiary DNS role after redeployment. `akoria.org` is the Active-Directory / Kerberos realm, authoritative on `radium` (Samba DC, BIND9_DLZ), mirrored hourly from `.net`. Hosts get an element name as their `A` record; functions are layered on as `CNAME`s (`dashboard`, `sso`, `git`, `db`, …). INFERRED: `solarian.net` appears only as branding — no zone for it exists in the repo.

SECTION 02

Control-plane architecture

Three C programs, one shared static library, and one binary protocol. Everything the fabric knows flows through `solariServer`; everything an operator does to it flows through a single local socket into the same process.

2.1 Three-tier topology & SCP transport

All three daemons link one static library, `libsolari` (`lib/solari/`), which owns framing, TLV, crypto, config, JSON, logging, time, the SQLite spool, and the nng transport. The core is zero-dependency; the transport (`solariNet.c`), spool (`solariSpool.c`), and reporter (`solariReporter.c`) compile only when their feature flags (`SOLARI_WITH_IO`, `SOLARI_WITH_SQLITE`) are set.

`solariClient` Per-host agent. Collects host metrics, log tails, and process info behind a platform-abstraction layer (only `platLinux.c` exists today); reports over SCP; spools to SQLite when offline. Discovery advertises via mDNS/avahi.

`solariMonitor` On-the-wire prober (`probeNet.c`: ICMP via unprivileged `SOCK_DGRAM` with fallback, TCP-connect, UDP, app-level). Targets are split across the monitor fleet by **rendezvous (HRW) hashing** at a replication factor — no coordinator. Linux-only.

`solariServer` The hub: validates and dispatches frames, persists to MariaDB, runs the alert engine, discovery enrichment, SNMP polling, provisioning, the leadership lease, and hosts the `solariCtl` operator bridge. Also builds `solariSnmpPoll` and `solariCtlClient` from the same core.

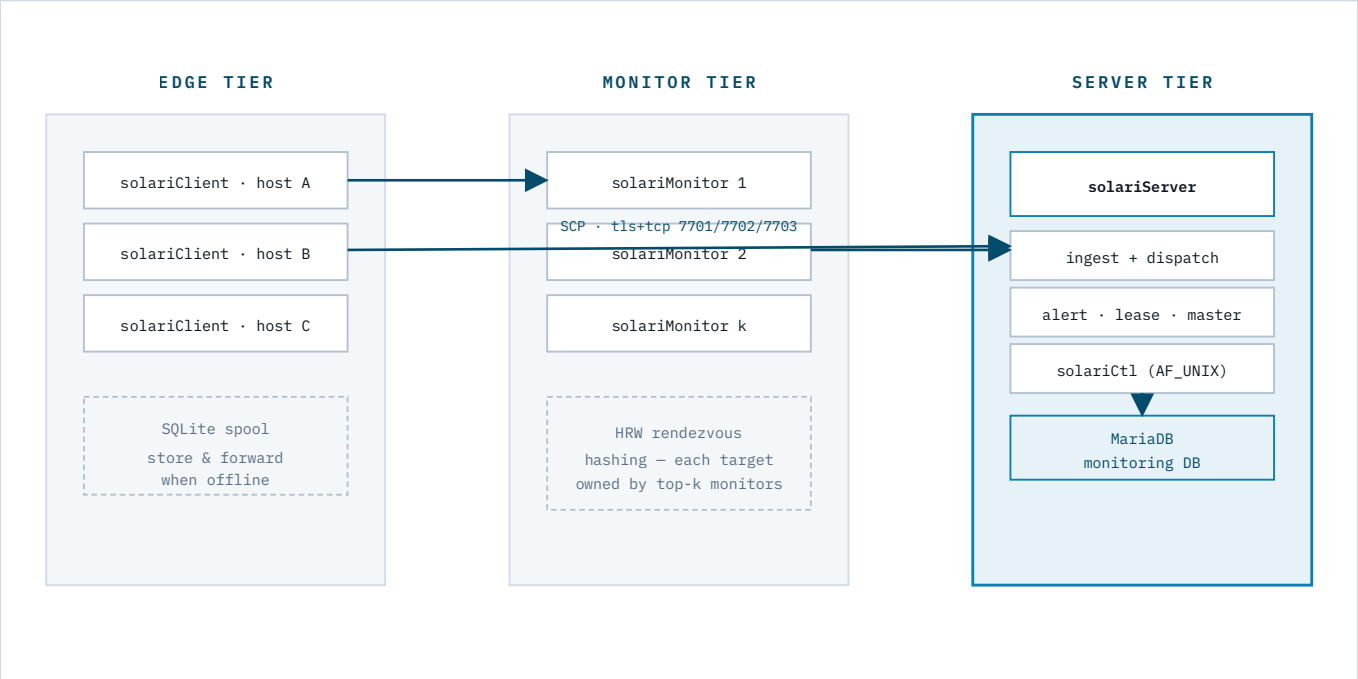


Figure 2.1 — Client and monitor tiers report over SCP (nng + mTLS) to the server, which persists to MariaDB. Monitor ownership is coordinator-free HRW. OBSERVED from src/ + include/solari/.

SCP frame & ingest gate

Every frame is validated by `solariFrameParse` (magic, version, CRC) before dispatch. The ingest path then derives the presenting node's **role from its TLS certificate CN** (`<role>.<fqdn>`), rejects any message type illegal for that role (`ERR_AUTH_ROLE`), and de-duplicates by `(sourceNodeId, seqNo)` via a process-local ring — best-effort, explicitly not durable exactly-once. Dispatch routes by message type:

SCP MESSAGE DISPATCH — SRC/SERVER/SERVERINGEST.C

MESSAGE	ROUTED TO	EFFECT
CLIENT_REPORT	serverDb + serverAlert	Host metrics persisted; host-scope alert rules evaluated
MONITOR_REPORT	serverDb + serverMaster + alerts	Probe results persisted; ownership reconciled
WATCHDOG	serverDbTouchNode	Liveness heartbeat
DISCOVERY_ADVERT	serverDiscovery	Discovered-host table updated
TOPOLOGY_REPORT	serverTopology	LLDP/topology edges
SURVEY_RESP / CONTROL_RESULT	serverControl	Command convergence tracking

2.2 solariServer internals & the solariCtl bridge

`solariCtl` is **not a separate binary** — it is a module compiled into `solariServer` (`src/server/solariCtl.c`) that listens on an `AF_UNIX` socket (default `/run/solari/solariCtl.sock`). It is the only way the dashboard mutates state. The protocol is line-oriented text:

```
# request / reply grammar (solariCtl.c)
request : VERB [k=v ...]\n
reply   : OK [k=v ...]\n  |  ERR <code> <message>\n
```

Destructive verbs (marked ⋆ in the verb table) require an `op=<operator>` attribution; `DECOMMISSION` additionally requires a two-step `confirm=<token>` handshake — the first call returns a token without acting. The same module holds the internal CA/CSR signing material (mbedTLS x509), so **the PHP dashboard never holds a server certificate or CA key**.

First-boot enrollment token exchange

The operator issues a 256-bit token with `solariCtlClient enroll-token`. Only its SHA-256 digest and expiry are stored in `enrollment`; the raw token is returned once to the operator. The target creates its private key locally and posts its CSR to `POST /api/enrollments` over HTTPS. This is an explicit bearer-authenticated public route: C rechecks the token digest, expiry, and prebound host and role before the CSR can become pending. The dashboard cannot issue tokens, sign CSRs, or write the monitoring database.

ENROLLMENT LIFECYCLE — MIGRATIONS 023–024 AND SOLARICTL

STATE / ACTION	AUTHORITY	DURABLE EFFECT
Issue token	Operator peer; `ENROLL_TOKEN`	Creates a host/role-bound row; stores digest and 15-minute default expiry
Submit CSR	Bearer token; `ENROLL_SUBMIT`	Moves token → pending; exact retries are idempotent by CSR fingerprint
Approve	Operator approval queue	One transaction stores approved node, decision, signed certificate, leaf DER SHA-256 fingerprint, and certificate serial; clears CSR
Pick up	Same bearer token	Repeated submission returns certificate + CA until the client acknowledges
Acknowledge	Same bearer token; `ENROLL_ACK`	Records certificate acknowledgement and clears the token digest
Reject	Operator approval queue	Marks rejected and clears the token digest and held CSR

Migration 024 stores the issued leaf fingerprint and serial on the node in the same approval transaction. Ingest currently compares the authenticated certificate CN with the source node ID and approved record; the installed NNG 1.x API does not expose peer DER for a live leaf-fingerprint comparison.

`deploy/enrollment/solari-enroll.sh` stages the returned PEM files, verifies the leaf against the returned CA, and checks that its public key matches the local private key before replacing installed material. Forced key renewal stays staged until the server returns a matching certificate; failure preserves the old key, leaf, and CA. If the certificate was installed but its server acknowledgement failed, `--resume` retries with the same token only when the server returns the exact installed leaf; it never replaces identity material during recovery. Tests reject a changed leaf and preserve the installed certificate. The DB lifecycle and installer are tested against a disposable database and a local mock endpoint. The full HTTPS → PHP → solariCtl → MariaDB → CA path is exercised by `tests/integration/test_enrollment_e2e.py` and runs in the full-server CI job.

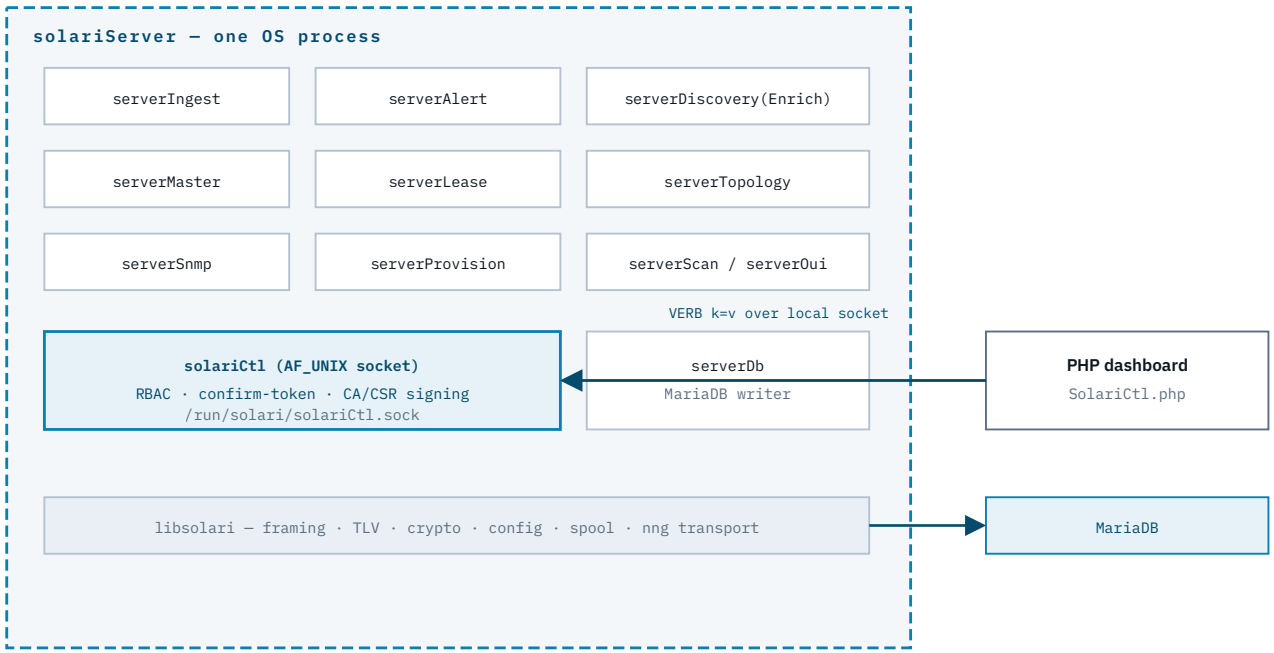


Figure 2.2 — Inside solariServer: the operator bridge solariCtl and the CA material live in-process; the dashboard reaches them only through the local socket. OBSERVED from src/server/.

Subsystem container views

Four subsystems carry most of the system's behaviour outside the C core: the PHP dashboard, the notification pipeline, the SoR change-data-capture path, and the discovery/enrichment loop. Each is drawn here as a nested container view — process boundary, the containers inside it, and the trust edge each one crosses.

3.1 The PHP dashboard

The dashboard is a single Apache + PHP-FPM application with **one front controller** (`dashboard/api/index.php`) and a custom `Router`. Every request passes one chokepoint — `Policy::enforce` — which is **default-deny**: a route with no explicit grant is refused, and every non-GET additionally clears a CSRF synchronizer-token + Origin check before the handler runs (Phase-0 hardening, commits `2d74adf` / `1812a9e`). The SPA is vendored React + Babel-standalone; **JSX is transpiled in the browser** at load. There is no bundler and no `package.json`. php-fpm runs as the unprivileged user `jason` and reaches the C core only through the `solariCtl` UNIX socket — it never holds a certificate or a DB super-grant.

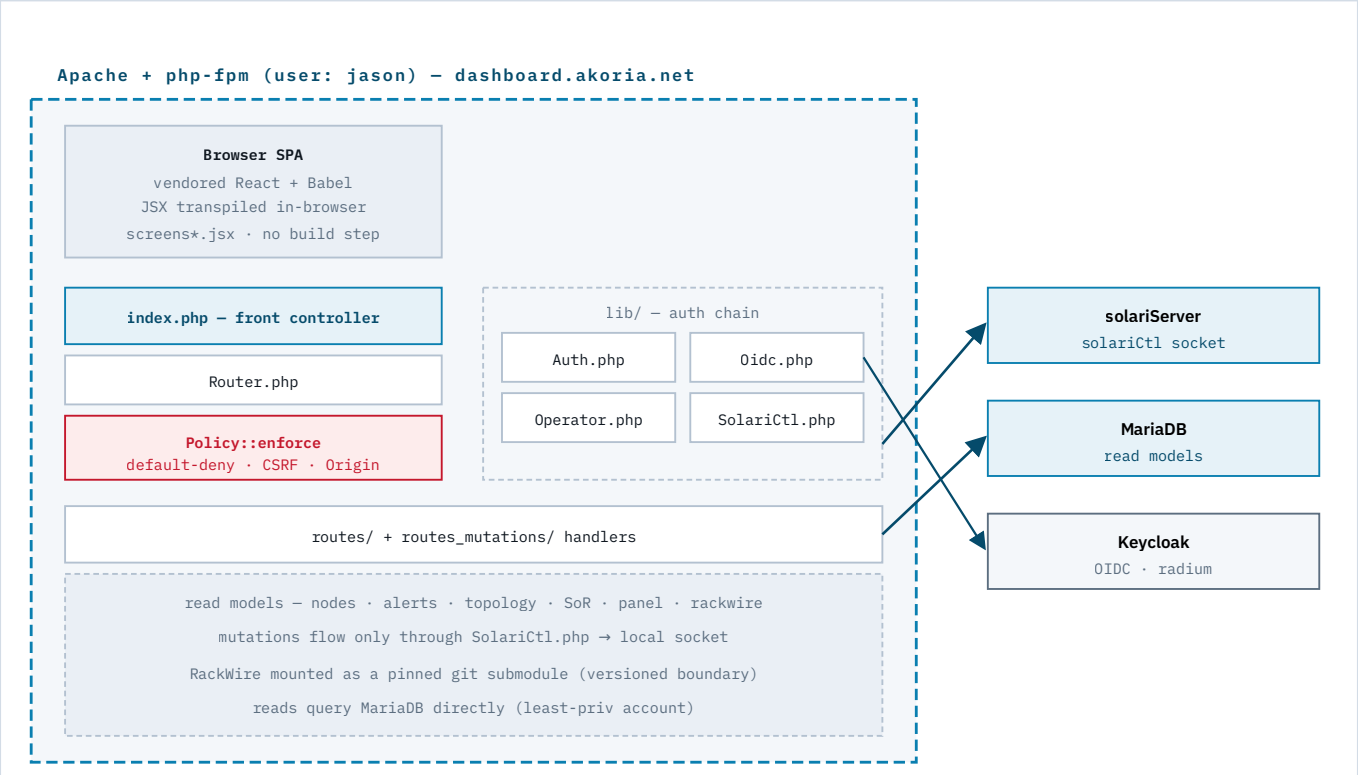


Figure 3.1 — Dashboard containers. One front controller, one default-deny policy chokepoint, mutations only via the solariCtl socket. OBSERVED from dashboard/.

3.2 Notification & SoR-CDC pipelines

Two Python daemons in `deploy/` bridge the fabric to the outside world over RabbitMQ. Neither is inside the C or PHP tiers — both are support glue, and both declare their exchanges defensively (there is no canonical broker config committed to the repo, a gap tracked below).

notify pipeline solariServer's alert engine publishes to the durable topic exchange `notify.events` (vhost `solari`, `amqp://…@10.5.2.50:5672/solari`). `notify.service` / `alertbridge.service` consume and fan out to delivery channels (Apple push bridge, etc.).

SoR CDC Writes to the SoR/CMDB DB fire `sor_outbox` triggers; `sor-emit` drains the outbox to the `sor.events` exchange; `sor-apply-dns` and `sor-apply-pihole` consume and reconcile DNS/Pi-hole. The reverse path (`sor-reconcile`) folds discovery back into the SoR.

> **KNOWN GAP — F-08**
>
> The current integration candidate enables publisher confirms for outbox delivery, requires mandatory routing before checkpointing, acknowledges consumer messages only after a successful apply, and reconciles missed state. These controls have local test coverage; production adoption and cross-lab review remain release gates. See the stabilization baseline for current evidence.

SECTION 04

Network topology & data flows

The fabric rides a flat 10.0.0.0/8 L2 domain segmented by convention into purpose-named /24s. This section maps the segments, then traces the three flows that matter: a client report, an operator mutation, and an SoR change propagating to DNS.

4.1 Network segmentation

OBSERVED ANOMALY — FLAT L2

OBSERVED on xenon: `ip route` shows the entire `10.0.0.0/8` as one on-link segment (`10.0.0.0/8 dev enp1s0 proto kernel scope link`). The /24 "subnets" below are an addressing convention, not enforced L2/L3 boundaries — xenon's ARP table holds neighbours across dozens of /24s outside the zoned set. Segmentation is administrative, not a security boundary.

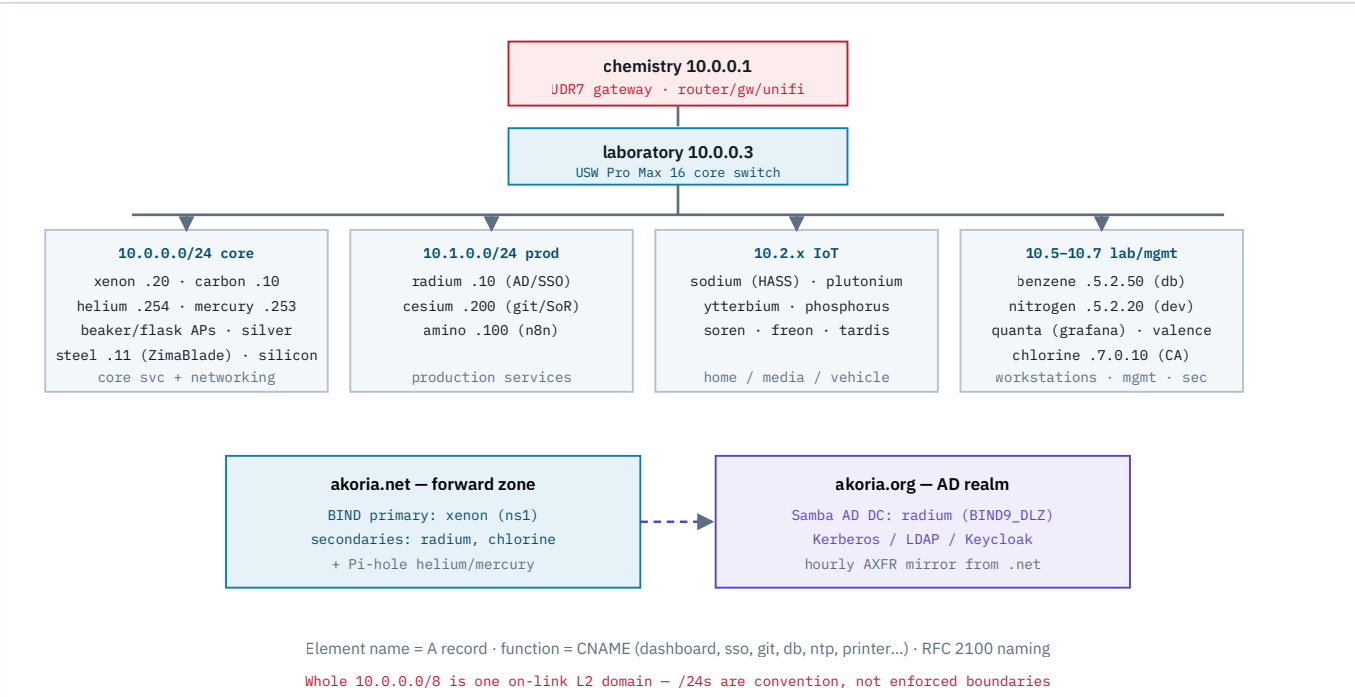


Figure 4.1 — Purpose-named /24 segments hanging off the chemistry→laboratory spine, and the two-domain DNS split. OBSERVED from zones + live route table.

4.2 Three canonical flows

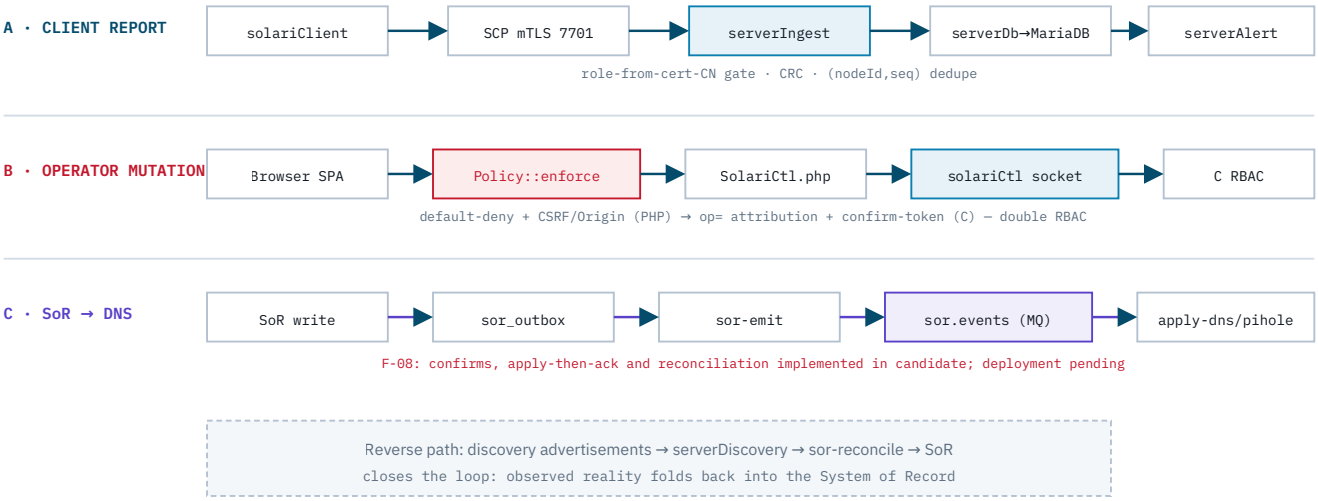


Figure 4.2 — The three flows that define the system: a report inbound, a mutation outbound through double RBAC, and an SoR change propagating to DNS. OBSERVED; candidate delivery controls remain pending deployment.

System inventory

Every host on the fabric, by name, manufacturer, type, and network address. Rows fuse the committed System of Record (`netdb/akoria-hosts.yml`) with a live discovery sweep from `xenon` captured on 2026-09-17. This historical snapshot has not been refreshed in this revision. Each row is flagged OBS where a fact was observed live and SoT where it comes only from the committed record.

AUTHORIZATION & METHOD

The discovery below is ordinary network administration on infrastructure the operator owns. Method: `nmap -sn -PR` host discovery across the 16 zoned /24s, offline IEEE OUI lookup against the local `nmap-mac-prefixes` table, and read-only SSH fact-gathering (`hostnamectl` / `dmidecode`) where a key was accepted. **No service/OS scan and no SNMP** completed — port/OS columns are honestly blank, not empty by absence of service. MACs are only shown where captured in the live ARP snapshot; none are synthesized.

5.1 Subnets

ADDRESSING SEGMENTS — 10.0.0.0/8 (ONE FLAT L2 DOMAIN; /24S ARE CONVENTION)

CIDR	PURPOSE	LIVE HOSTS UP	FLAG
10.0.0.0/24	Core networking + core services	chemistry, laboratory, carbon, xenon, beaker, flask, silver, catalyst, accelerant (+ 5 un-named)	OBS
10.0.1.0/24	Workstation + recovered/legacy (boson, muon, gallifrey, hydrogen)	none up at scan	OBS
10.1.0.0/24	Production — AD/DC, Forgejo/SoR, n8n	radium, amino, cesium	OBS
10.2.0.0/24	IoT — Home Assistant, voice	sodium, plutonium	OBS
10.2.2.0/24	IoT — media / console	ytterbium, phosphorus	OBS
10.2.3.0/24	IoT — appliance, water dish	soren	OBS
10.2.4.0/24	IoT — thermostat	freon	OBS
10.3.0.0/24	Peripherals — radio, scanner, UV printer	none up at scan	OBS
10.4.1.0/24	Personal — tablet (callisto)	none up at scan	OBS
10.4.4.0/24	Personal — vehicle (tardis)	tardis	OBS
10.5.0.0/24	Workstations/SBC — tungsten	none up at scan	OBS
10.5.2.0/24	Workstations/SBC — nitrogen, benzene, astatine	nitrogen, benzene	OBS
10.5.87.0/24	Lab gear — eleksmaker laser/CNC	none up at scan	OBS

CIDR	PURPOSE	LIVE HOSTS UP	FLAG
10.6.0.0/24	Network mgmt — Grafana, control panel, outpost	valence	OBS
10.6.6.0/24	Network mgmt — GPS/PTP time source	none up at scan	OBS
10.7.0.0/24	Security — NVR hub, internal CA	photon, chlorine	OBS
203.0.113.0/24	RFC 5737 documentation range — host disc is a placeholder, not real infra	not scanned	SoT

Reverse zones 10.1.4/24, 10.2.5/24, 10.3.2/24, 10.4.2/24 are declared in BIND but hold no hosts in the current SoT — stale zones or an incomplete host list. OBS

5.2 Host inventory — infrastructure & services

CORE INFRASTRUCTURE HOSTS. MAC = LIVE ARP CAPTURE; HARDWARE CONFIRMED BY SSH WHERE NOTED.

HOST	IP	MANU-FACTUR-ER	TYPE / MODEL	ROLE · KEY CNAMES	MAC (OBS)	FLAG
chem-istry	10.0.0.1	Ubiqui-ti	UDR7 Dream Router 7	gateway/firewall/UniFi · router,gw,unifi,ns0	74:fa:29:41:f0:57	OBS↑
labo-ratory	10.0.0.3	Ubiqui-ti	USW Pro Max 16	core switch (etherlighting)	9c:05:d6:e4:f1:ac	OBS↑
beaker	10.0.0.7	Ubiqui-ti	U7 Pro	WiFi 7 AP	a8:9c:6c:4c:9a:d0	OBS↑
flask	10.0.0.8	Ubiqui-ti	U7 Pro	WiFi 7 AP	a8:9c:6c:6a:09:c0	OBS↑
xenon	10.0.0.20	youyee-too	X1 SBC · Celeron N5095 · 16 GiB · Ubuntu 26.04	SolariNet serv-er+dashboard+BIND pri-mary · dash-board,monitor,solari,ns1	(self)	OBS✓ssh
carbon	10.0.0.10	UGREEN	DXP4800 **Plus** · Debian 12 · box-hostname "NAS-X"	NAS · nas,nas-x,c	6c:1f:f7:0c:76:0f	OBS✓ssh
helium	10.0.0.254	Rasp-berry Pi	Raspberry Pi (model unspec.)	Pi-hole primary/resolver · ns2,hole,dns1,he	—	SoT
mer-cury	10.0.0.253	Ice-Whale	Zima (model unveri-fied)	Pi-hole secondary · ns3,hole2,dns2. Operator confirms three Zima units total (Blade, Board, Board 2); physi-cal mapping pending.	—	SoT / operator correcti

HOST	IP	MANU-FACTUR-ER	TYPE / MODEL	ROLE · KEY CNAMES	MAC (OBS)	FLAG
steel	10.0.0.11	Ice-Whale	ZimaBlade	Inventory-labeled Steel; on port 12 of laborato-ry. Booted without use-ful services; currently unreachable. Not adver-tised as a nameserver. Planned tertiary DNS role follows console access and redeployment.	—	operator correction
sili-con	10.0.0.250	Rasp-berry Pi	Raspberry Pi 4	edge proxy (Caddy auto-ACME)	—	SoT
scan-dium	10.0.0.12	Ice-Whale	Zima (model unveri-fied)	services (n8n/Uptime-Kuma) — offline/planned. Physical Zima model mapping pending.	—	SoT / operator correcti
halo-gen	10.0.0.2	Ubiqui-ti	UGC Max	old primary router (offline, minirack pending)	—	SoT
radium	10.1.0.10	Rasp-berry Pi	CM5 · Debian 13 · arm64	Samba AD DC + Keycloak + BIND9_DLZ/.org · ad,sso,ldap,kdc	2c:cf:67:b5:ab:7d	OBS✓ssh
cesium	10.1.0.200	Dell	PowerEdge R410 · openSUSE Tumble-weed	Forgejo + SoR/CMDB MariaDB · git,ad-backup	d4:ae:52:97:44:e9	OBS✓ssh
amino	10.1.0.100	Ice-Whale	ZimaBoard 832 · ZimaOS · Celeron N3450 · 8 GiB	n8n workflow automation · n8n	00:e0:4c:17:7c:97	OBS✓ssh
ben-zene	10.5.2.50	HP	EliteDesk 800 G6 Mini · i5-10500T · open-SUSE	MariaDB (SoR mirror) + fleet netboot :8080 · db	e0:70:ea:b0:13:e6	OBS✓ssh
nitro-gen	10.5.2.20	Rasp-berry Pi	Pi 500 · Debian 13 · box-hostname "dev-family-pi01"	dev / IDE · dev	2c:cf:67:bc:d5:2e	OBS✓ssh
tung-sten	10.5.0.5	Minis-forum	MS-R1	AI/dev workstation (AD-joined)	—	SoT

HOST	IP	MANU-FACTUR-ER	TYPE / MODEL	ROLE · KEY CNAMES	MAC (OBS)	FLAG
asta-tine	10.5.2.10	Argon41	OneUp SBC	SBC workstation · laptop	—	SoT
quanta	10.6.0.10	Rasp-berry Pi	Pi 5	Grafana · grafana	—	SoT
va-lence	10.6.0.20	Friend-lyElec	NanoPi-R1 (armhf)	monitoring outpost	02:81:12:d7:eb:db (locally-admin MAC)	OBS↑
chlo-rine	10.7.0.10	Khadas	VIM4 · Ubuntu 22.04 · arm64	internal CA/key server (step-ca) + BIND 9.18.39 authoritative secondary for akoria.net and gen-erated reverse zones; all 18 SOA serials matched xenon on 2026-10-21. Bare zones and RPZ are not served here.	c8:63:14:72:87:81	OBS✓SSH / DNS
tachy-on	10.6.6.10	Parti-cle	Tachyon	GPS/PTP time source · ntp	—	SoT

OBS↑ = up + MAC observed live; OBS✓ssh = hardware confirmed by SSH fact-gather; SoT = committed record only, host down or unreachable at scan.

5.3 Host inventory — endpoints, IoT & peripherals

CLIENT / IOT / PERIPHERAL HOSTS (LARGELY SOT; MAC WHERE LIVE-OBSERVED).

HOST	IP	MANU-FACTUR-ER	TYPE / MODEL	ROLE	MAC (OBS)	FLAG
silver	10.0.0.85	Epson	ET-8550	printer · printer	64:c6:d2:5b:c3:2a	OBS↑
cata-lyst	10.0.0.90	Hitron	MoCA adapter	Family Room MoCA bridge	dc:36:0c:d5:db:7d	OBS↑
accel-erant	10.0.0.91	Hitron	MoCA adapter (inferred from OUI)	unknown ("recov-ered from DNS")	dc:36:0c:d6:11:b5	OBS↑
hydro-gen	10.0.1.50	Apple	Mac Mini M4 Pro	primary worksta-tion	—	SoT
sodium	10.2.0.20	—	unknown	Home As-sistant · hass	88:a2:9e:24:d3:dc (Raspberry Pi OUI)	OBS↑
pluto-nium	10.2.0.100	Amazon	Echo Dot	voice as-sistant	f8:54:b8:d2:ac:76	OBS↑

HOST	IP	MANU-FACTUR-ER	TYPE / MODEL	ROLE	MAC (OBS)	FLAG
ytter-bium	10.2.2.2	Sony	PlayStation 5	media / console	78:c8:81:c1:04:b6	OBS↑
phos-phorus	10.2.2.10	Google	Google TV 4K	media (Family Room)	b8:7b:d4:ff:ce:6d	OBS↑
soren	10.2.3.10	Espres-sif (OUI)	LR4 MBR appliance	MBR ap-pliance	c4:d8:d5:6e:32:c0	OBS↑
dihy-drox-ide	10.2.3.20	—	Pawaii	smart wa-ter dish	—	SoT
freon	10.2.4.10	Google/ Nest	Nest thermostat	thermo-stat	3c:31:74:dd:3f:87	OBS↑
tardis	10.4.4.10	Tesla	Model 3	vehicle	cc:88:26:26:f3:db (LG Innotek OUI)	OBS↑
cal-listo	10.4.1.10	Apple	iPad Pro 12.9	tablet	—	SoT
photon	10.7.0.1	Eufy	Eufybase NVR	NVR hub	04:17:b6:88:66:75	OBS↑
bro-mide	10.3.0.80	Epson	RR-600W	scanner · scanner	—	SoT
plat-inum	10.3.0.100	Eufy-Make	E1 UV printer	UV print-er	—	SoT
radio	10.3.0.34	—	"Radio Pi"	radio backend	—	SoT
eleks-maker	10.5.87.15	Eleks-Maker	laser/CNC con-troller	lab tool	—	SoT
bo-son / muon	10.0.1.125/.126	—	ArkKVM	KVM units · arkkvm1/2	—	SoT
lithi-um	10.6.4.33 / 10.6.3.33	Arduino	Imola · Debian 13	status-panel host ⚠ on **unzoned** subnets	—	OBS✓ssh

RECONCILIATION QUEUE — OBSERVED DRIFT & UNKNOWNS

- **Five un-named live hosts** answered ARP in 10.0.0.0/24 with no PTR and no SoT record: `10.0.0.6, .31, .32, .33, .160`. No MAC captured (cache aged out). Candidate unlabeled or rogue devices — worth identifying. OBS
- **lithium** sits on `10.6.3.0/24` and `10.6.4.0/24`, neither of which is a zoned subnet — a real SoT host on unadvertised address space. OBS
- **steel / iron handoff**: live SoR and DNS still assign `.11` to `steel`, while `iron` is reserved for the post-redeployment ZimaBlade and tertiary DNS role. The operator confirms the Steel inventory label is on the ZimaBlade; its switch location is laboratory port 12. Keep the live name/IP assignment until console access and redeployment complete. OBS
- **nitrogen** DNS name vs. box hostname `dev-family-pi01`; SoT lists it as a Pi 500, live kernel is a CM5-class 2712 — confirm the exact model. OBS
- **"Recovered from DNS"** hosts (quartz, tin, accelerant, bunsen, gallifrey, strangeness) have genuinely unknown hardware — reconstructed from production DNS, never deliberately assigned. SoT

SECTION 06

Component reference

The runnable parts of the system — daemons, support services, and the ports and sockets that connect them.

6.1 Binaries (C tier)

COMPILED FROM THE SINGLE CMAKE C99 TREE; ALL LINK LIBSOLARI.

BINARY	RUNS ON	LISTENS / CONNECTS	PURPOSE
`solariServer`	xenon	SCP `tls+tcp` 7701–7703 in; `solariCtl.sock`; MariaDB out	Ingest, persist, alert, discovery, SNMP, lease/master, operator bridge, CA signing
`solariClient`	fleet hosts	SCP out to server; SQLite spool local; mDNS advertise	Per-host metrics/logs/process reporting with offline store-and-forward
`solariMonitor`	designated hosts	SCP out; ICMP/TCP/UDP/app probes to targets	On-the-wire reachability probing; HRW-partitioned target ownership
`solariSnmpPoll`	xenon	SNMP out; SCP in-process	SNMP metric collection (built from the same core)
`solariCtlClient`	any	`solariCtl.sock`	CLI to the operator bridge for scripts/tests

6.2 Services & systemd units

DEPLOYED UNITS — DEPLOY/HOST-CONFIGS/XENON/*.SERVICE AND THE WIDER FABRIC.

SERVICE	HOST	ENDPOINT	NOTES / FLAG
SolariNet dashboard	xenon	dashboard.akoria.net (HTTPS vhost, :9443 intranet)	Apache + php-fpm (user jason) OBS
`solarinet-server / -monitor / -snmp`	xenon	systemd units	C daemons; hardened units OBS
`notify / alert-bridge`	xenon	consume `notify.events`	alert fan-out to delivery bridges OBS
`sor-emit / -apply-dns / -apply-pi-hole / -reconcile`	xenon (live units)	RabbitMQ `sor.events`	delivery controls implemented in candidate; deployed configuration still differs flag
BIND (akoria.net)	xenon primary; radium + chlorine secondary for forward; chlorine secondary for reverse	:53	xenon also serves RPZ and 90 split-horizon bare zones; chlorine does not yet replicate those OBS
Samba AD DC + Keycloak	radium	LDAP/Kerberos/OIDC; sso.akoria.net	authoritative akoria.org OBS

SERVICE	HOST	ENDPOINT	NOTES / FLAG
Forgejo	cesium	git.akoria.net	STATIC_ROOT_PATH on internal storage SoT
MariaDB — SoR/ CMDB	cesium (primary), benzene (mirror)	db.akoria.net → benzene	separate DB from monitoring SoT
RabbitMQ	benzene	`amqp://… @10.5.2.50:5672/solari`	notify.events + sor.events; no canonical broker config in repo flag
step-ca internal CA	chlorine	ca/acme/pki.akoria.net	X.509 + S/MIME + ACME + SSH CA SoT
Fleet netboot HTTP	benzene	:8080	staged, inert — contained (F-07) flag

6.3 Ports & sockets

7701 / 7702 / 7703 SCP over `tls+tcp` (nng + mbedTLS mTLS) — client, monitor, and control channels into solariServer.

/run/solari/solariCtl.sock AF_UNIX line protocol — the only mutation path from the dashboard to the C core.

9443 / 443 Dashboard HTTPS (intranet additive vhost / public subdomain vhost).

53 DNS — BIND on xenon and radium; chlorine serves secondary akoria.net/reverse zones; Pi-hole on helium/mercury. Steel is offline and not advertised.

5672 RabbitMQ AMQP on benzene (vhost `solari`).

SECTION 07

Data model & System of Record

Two MariaDB databases, deliberately separate: a monitoring datastore that records what the fabric observes, and a System of Record that holds what the fabric is supposed to be. Discovery flows from the first into the second; DNS and Pi-hole flow out of the second.

7.1 Two databases

Monitoring DB 32 application InnoDB tables maintained by 28 SQL migration files (numbered `001–027`, with two distinct `016_*` migrations), plus the migration ledger — nodes, reports and receipts, probe results, alerts and rules, topology edges, discovery, panel/criticality, lifecycle tombstones, enrollment identity, and durable decommission challenges. All timestamps stored UTC.

SoR / CMDB Separate DB on `cesium`, mirrored to `benzene`. The canonical entity spine is defined in `netdb/sor/schema.sql` (`netdb/sor/migrations/` holds only incremental deltas): `entities, hardware_models, hardware_units, locations, networks, subnets, vlans, interfaces, ip_addresses`. Change-data-capture uses `sor_outbox` and `sor_sync_state` from ledgered migration `022_transactional_outbox.sql`, with the 21 SoR CDC triggers in migration `023_sor_cdc_triggers.sql`; the former deploy SQL is retained only as a legacy reference.

F-12 STATUS — CLEAN INSTALLS REPRODUCIBLE; LEGACY ADOPTION OPEN

`tools/migrate.py` applies the monitoring and SoR streams with immutable full-file IDs, SHA-256 checksums, a durable ledger, ordered gap detection, and explicit refusal of unknown legacy schemas. `tools/export_schema.py` rebuilds both streams in disposable databases and exports `db/schema.sql` and `netdb/sor/schema.snapshot.sql` with their migration manifests. CI checks the generated snapshots byte-for-byte. The two monitoring files numbered 016 have distinct ledger IDs and deterministic order.

The generated snapshots are reference artifacts; install and upgrade through the ledgered runner. Existing production databases have no migration ledger, so backup, full schema comparison, and reviewed history adoption remain release gates. `solariServer --init-db` is retired because it cannot provide that safety.

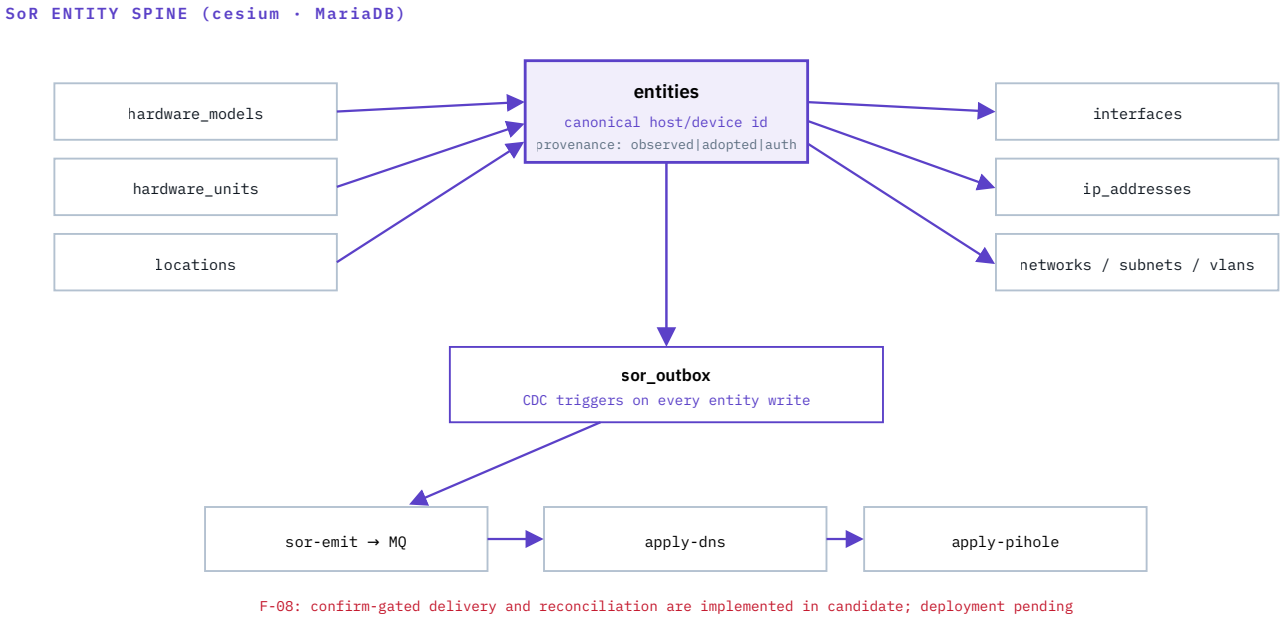


Figure 7.1 — The SoR entity spine and its CDC outbox. Every page in the planned integration (I2/I4) hangs off entities. OBSERVED from netdb/sor/schema.sql + deploy/sorsync/sql/.

7.2 Provenance as a first-class column

The SoR distinguishes how it learned each fact: `observed` (seen by discovery), `adopted` (an operator promoted an observation to intended state), and `authoritative` (declared, the source of truth others reconcile to). The planned I2 track surfaces this provenance on every entity page, so the dashboard can never again present a discovered guess as a declared fact — the same honesty discipline that governs this document's OBS/INF flags.

SECTION 08

Identity, authorization & trust boundaries

Authorization is enforced twice on the dangerous path — once in PHP before a command is formed, and again in C before it executes — and identity is derived from certificates on the wire, not from claims in a payload.

8.1 The human auth chain

A browser request is authenticated and authorized through four cooperating libraries before any handler runs. Local accounts use bcrypt; federated login uses Keycloak's OIDC authorization-code flow (confidential client), with Keycloak in turn federated to Samba AD over LDAPS. The legacy direct-LDAP path is a **fail-closed stub** — it denies rather than degrading open.

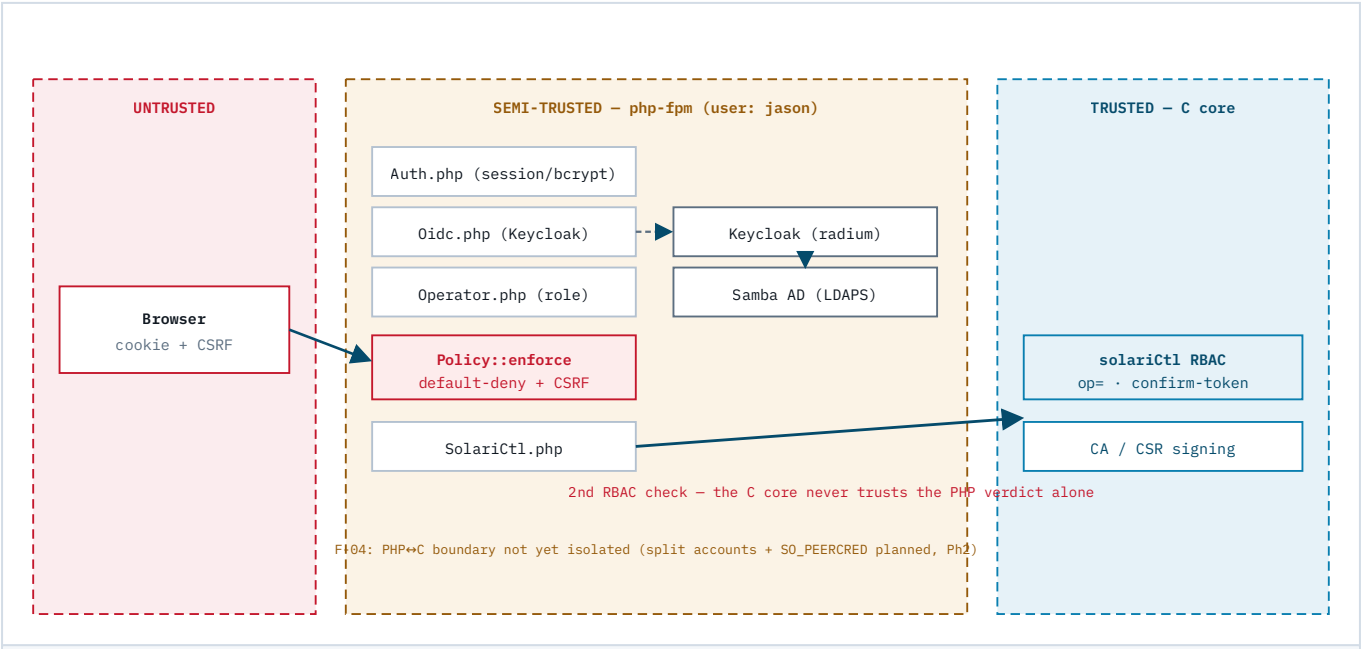


Figure 8.1 — Three trust zones and the double authorization check. Identity federates browser → Keycloak → AD; enforcement happens in PHP and again in the C core. OBSERVED from dashboard/api/lib/ + src/server/solariCtl.c.

8.2 Machine identity on the wire

Machines do not authenticate with claims. Every SCP peer presents a client certificate whose CN encodes `<role>.<fqdn>`; the ingest path derives the role from the verified certificate and rejects any message type illegal for that role (`ERR_AUTH_ROLE`). The planned F-01 work makes the binding total — cert SAN/CN ↔ node ID ↔ role ↔ FQDN as one canonical identity contract, so a valid certificate can never be replayed under a different node's identity.

8.3 Trust-boundary summary

WHERE TRUST CHANGES HANDS — AND THE FINDING THAT HARDENS EACH EDGE.

BOUNDARY	CONTROL TODAY	HARDENING	SEV
Browser → PHP	session + default-deny Policy + CSRF/Origin	claims validation, break-glass disable, rate-limit, role revalidation (F-14)	P1
Node bootstrap → PHP	host/role-bound bearer digest + expiry; C verifies before CSR storage	real HTTPS → Ctl → DB → CA integration test; rate-limit public bootstrap	P1
PHP → C (solariCtl)	local UNIX socket; op= attribution; confirm-token for destructive verbs	framed I/O + deadlines + caps (F-11); split least-priv accounts + SO_PEERCRED (F-04)	P0
Node → server (SCP)	mTLS; role-from-cert-CN; illegal-type reject	frame↔cert identity binding (F-01); mTLS invariant not configurable (F-29)	P0
Server → DNS/AD	SoR-driven render + apply	render+validate+atomic-switch+rollback (F-17); AD/LDAP write paths frozen	P1

SECTION 09

Developer workflow & build

One CMake tree builds the C tier; the dashboard has no build at all. The house rules are strict and load-bearing — they are why the system can be reasoned about.

9.1 Building the C tier

The control plane is ISO C99 built with CMake, wrapped by a thin top-level `Makefile` for convenience. Optional subsystems compile behind feature flags, so a minimal client can be built without the SQLite spool or the nng transport.

```
# from repo root — the Makefile just drives CMake
make            # configure + build all binaries into build/
make test       # run the C test suite (golden SCP frames, unit)

# feature flags (CMake cache):
SOLARI_WITH_IO=ON        # nng transport + reporter (solariNet.c)
SOLARI_WITH_SQLITE=ON    # client-side durable spool (solariSpool.c)
```

Wire compatibility is protected by **golden-frame fixtures**: canonical SCP frames are committed per protocol version and are immutable — a change that alters the bytes of an existing version fails the test rather than silently breaking older peers.

9.2 The dashboard has no build step

HARD HOUSE RULE

There is **no bundler, no `package.json`, no `composer.json`, and no Node.js** anywhere in the runtime. OBSERVED — none of those files exists in the tree. React and Babel are vendored under `dashboard/public/`; JSX in `screens*.jsx` is transpiled in the browser at load. To develop the UI you edit a `.jsx` file and reload — there is nothing to compile. Introducing a build step into this code is forbidden by project policy, not merely discouraged.

9.3 Coding standards

Language	Standard C (K&R/ANSI) for the application; functional PHP 8+ for the dashboard; browser-only JS. **No server-side JavaScript, ever.**
Naming	`camelCase` is the house standard for variables, functions, and named structures across C and PHP alike — including where the language community would use snake_case.
Paradigm	Prefer functions to objects. Classes must earn their place with real shared-state invariants; the C core uses an opaque-handle pattern, not OOP emulation.
Comments	Every program, function, and non-obvious control structure carries a brief purpose / input / output statement. Explain intent and edge cases, not the obvious.

9.4 Tests & the CI gates being added

Dashboard authorization has PHP tests (`tests/dashboard/test_authz_enforce.php`, `test_authz_dispatch.php`) exercising the default-deny matrix and the request-method-to-verb dispatch. There is no `phpunit` or `CONTRIBUTING` in the tree yet. The stabilization program adds a battery of CI gates: a live-DB gate (`SOLARI_TEST_DB=1`), TLS-loopback + cert-identity failure tests, a generated route×method×role matrix with CSRF/Origin tests, submodule init before dashboard tests, migration clean-install / upgrade / idempotent-rerun checks, shellcheck/ASAN/UBSAN/fuzzers, and fault injection (broker loss, lease race, partial I/O, backup restore). A **cross-lab review gate** is required before committing to Phase-2 architectural choices — the original review was single-lab.

SECTION 10

Operations & deployment

SolariNet is deployed as hardened systemd units driven from the `deploy/` tree. This section covers how it runs, and — just as important — which capabilities are deliberately switched off.

10.1 Deployment layout

`deploy/host-configs/`	Per-host systemd units and Apache vhosts (e.g. `xenon/solarinet-server.service`, `apache-solarinet.conf`). Units are hardened (namespacing, restricted syscalls).
`deploy/fleet/`	Ansible inventory + a netboot `catalog.json`. The netboot HTTP service on `benzene:8080` is staged and inert — see freeze below.
`deploy/dns-bare/, netdb/`	Zone generation from the SoT: `gen-zones.py` renders `akoria.net`; `sync-net-to-org.py` mirrors into the AD realm hourly; `gen-bare.py` produces the ~100 split-horizon per-host zones.
`deploy/sorsync/`	The CDC units — `sor-emit`, `sor-apply-dns`, `sor-apply-pihole`, `sor-reconcile` — running on xenon.
`status-panel/`	The Galactic Unicorn firmware, shared USB/TLS wire protocol, and lithium daemon. The subproject's developer reference covers its module boundaries and test workflow; operator behavior is in the panel user guide.

10.2 Frozen capabilities

FEATURE FREEZE — PHASE 0 CONTAINMENT

The following are implemented but held `LAB`-only until their Phase-blockers close. They must not be enabled in production, and no new features may be built until Phase 4 gates pass.

- **Tab5 approvals** — approval display not cryptographically bound to the approval (F-06/F-24).
- **One-button PXE provisioning** — executes unauthenticated artifacts as root; key material left staged (F-07/F-16).
- **Arbitrary MCP SQL** — replaced with fixed tools/views + a read-only account (F-18).
- **Opie automation** — "read-only" model is not confidentiality-safe; sandboxed (F-15).
- **Remote CA automation** — kept manual on `chlorine`.
- **LDAP/AD write paths** beyond current use — frozen; AD is read-federated only.

10.3 Operational cautions

BEFORE YOU ACT

- **Alarm tests are loud.** The status-panel and alert paths drive real physical notification (LED panel on `lithium`, push bridges). A test alarm rings the actual fabric — announce it first.
- **Destructive verbs need attribution.** `DECOMMISSION`, `RETIRE`, `SIGN`, `DEPLOY`, `FLEET_*`, and the `*_REMOVE/_DEL` verbs require `op=`; `DECOMMISSION` needs the two-step confirm-token.
- **Backups are not proven restorable.** F-27 — nonempty ≠ restorable; a restore drill is Phase-3 work. Do not assume a backup will roll back a bad change.
- **DNS/AD apply has no rollback yet.** F-17 — apply paths lack atomic switch + rollback semantics; a bad zone render is not automatically reversible today.

10.4 Health philosophy

The system's guiding operational rule, learned from the 2026-07-07 cesium incident (Forgejo served 500s for ~26 hours while a bare TCP check stayed green): **a socket accepting a connection is not health.** The host-health contract requires application-level liveness, and the planned freshness envelope `{lastSuccess, ageSeconds, state}` (I3) makes every page state its own staleness rather than presenting cached data as live.

SECTION 11

Planned architecture & the stabilization program

An independent architecture review found that feature breadth had outrun enforcement and durability. The response is a declared feature freeze and a phased program with two goals: make the system honest and make it whole. Nothing new ships until it is done.

11.1 Two goals

Stability "Done" means authorized and durably persisted and correctly delivered and actually applied and currently fresh — not merely "a socket accepted a message."

Integration One entity spine, one freshness/health contract everywhere, one authorization matrix, consistent cross-navigation — pages and services tightly interlinked rather than parallel silos.

11.2 Phase roadmap

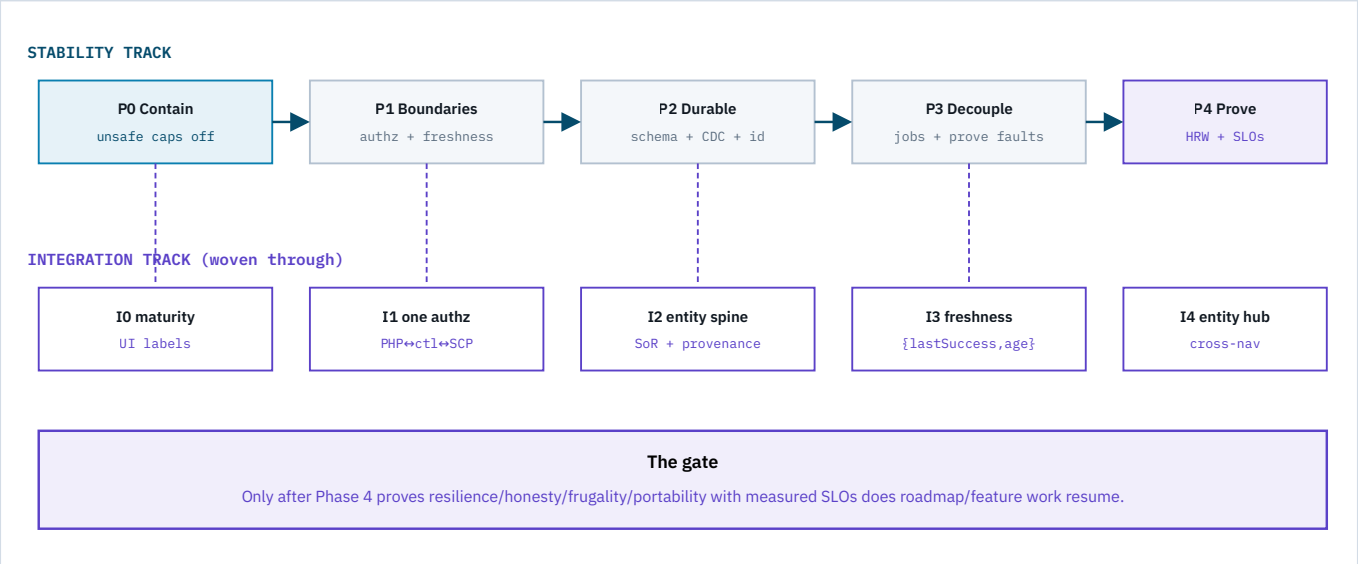


Figure 11.1 — Two tracks, five phases. Stability hardens the plane; Integration unifies the surface. OBSERVED from the stabilization workflow doc (2026-08-09).

11.3 What is real vs. designed today

CAPABILITY MATURITY — THE DISTINCTION THIS DOCUMENT EXISTS TO PRESERVE.

CAPABILITY	MATURITY	NOTE
Client reporting + ingest + MariaDB persistence	PRODUCTION	load-bearing across the fleet
Dashboard authz (default-deny + CSRF)	PRODUCTION	Phase-0 hardening landed (2d74adf/1812a9e)
SoR + CDC → DNS/Pi-hole	LAB	works; can lose events (F-08) until Phase-2

CAPABILITY	MATURITY	NOTE
Monitor tier + HRW k-of-n redundancy	PROTOTYPE	coded, not load-bearing; nearly all nodes are client-role (F-19/F-20)
Two-server HA / lease failover	PROTOTYPE	production is fleet-of-one; prove in Phase-3/4 (F-09)
Fleet PXE provisioning	LAB	frozen — unauthenticated-as-root (F-07)
Decommission lifecycle	LAB	script exists; not first-class/dashboard-driven yet

11.4 Findings register — F-01 … F-31

The complete register from the 2026-08-09 review, with severity and program disposition. This is the authoritative backlog; each item is Adopted (fix as designed), Adapted (fix to the real fleet-of-one), Contained (kept lab-only now, fixed later), or Folded (merged into another track).

FINDINGS F-01..F-31 — FINDINGS & RECOMMENDATIONS 2026-08-09 · DISPOSITIONED IN THE STABILIZATION WORKFLOW

ID	TITLE	SEV	DISPOSITION
F-01	Node certs not bound to frame identity	P0	Adopt · I1 / Ph2
F-02	Dashboard authorization systemically incomplete	P0	Adopt (default-deny) · I1 / Ph1 ✓
F-03	Cookie-auth control requests have no CSRF defense	P0	Adopt (token+Origin) · Ph1 ✓
F-04	PHP/C trust boundary not isolated	P0	Adopt (split accounts, SO_PEERCRED) · Ph2
F-05	Viewer-triggerable scan can starve the lease	P0	Adopt · gate Ph1, job-runner Ph3
F-06	Tab5 approval not cryptographically bound	P0	Contain now · Ph0
F-07	PXE runs unauthenticated artifacts as root	P0	Contain now, signed-manifest Ph3
F-08	SoR CDC can permanently lose events	P1	Adopt (confirms+reconcile) · I2 / Ph2
F-09	Server HA semantics internally inconsistent	P1	Adapt (fleet-of-one) · prove Ph3/4
F-10	Health & convergence records can lie	P1	Adopt (persist once, fsync) · I3 / Ph2
F-11	solariCtl wire protocol unsafe both ends	P1	Adopt (framed I/O+caps) · Ph1
F-12	Schema lifecycle not reproducible	P1	Adopt (one migrator, ledger) · I2 / Ph2

ID	TITLE	SEV	DISPOSITION
F-13	Browser freshness violates honesty goal	P1	Core and per-source cache/stale/SSE mitigations in candidate; finish authenticated browser checks · I3 / Ph1
F-14	Auth/session controls incomplete	P1	Adopt (claims, rate-limit) · Ph1
F-15	Opie "read-only" not confidentiality-safe	P1	Contain now, harden Ph3
F-16	Fleet provisioning leaves key material staged	P1	Adopt (node keys, one-time CSR) · Ph3
F-17	DNS/AD apply lacks safe deploy semantics	P1	Adopt (atomic+rollback) · I2 / Ph2–3
F-18	Arbitrary MCP SQL not safely constrained	P1	Contain (fixed views, RO acct) · Ph0
F-19	Monitor probing sequential/capped	P2	Adopt as batch · Ph3–4
F-20	HRW membership unproven; claim too broad	P2	Adopt + doc-fix now · Ph3–4
F-21	Spool has no quota/age; sequence reuse	P2	Adopt as batch · Ph3–4
F-22	SoR multi-step writes untransacted	P2	Adopt · folded into Ph2 (F-17)
F-23	Decommission receiver and delivery	P2	Challenge gate, target-node field, generation-bound acknowledgement, and persistent bounded delivery retry implemented; client receiver and control-queue token redaction remain open
F-24	Tab5 secret-at-rest unproven	P2	Adopt as batch · Ph3–4
F-25	UniFi false-green	P2	Fold into I3 · pulled into Ph1
F-26	Authbroker fails open on empty token	P2	Adopt as batch · Ph3–4
F-27	Backups nonempty ≠ restorable	P2	Adopt as batch · Ph3–4
F-28	Sparse referential integrity / orphans	P2	Adopt as batch · Ph3–4
F-29	mTLS configurable, not invariant	P2	Adopt as batch · Ph3–4
F-30	Warning-clean CI + sanitizers + fuzzing	P3	Adopt into CI gate · ongoing
F-31	CSP/HSTS/headers under no-build constraint	P3	Adopt into CI gate · ongoing

✓ = implemented on the local integration candidate named in the cover, based on the current audit. A checkmark does not mean the change has been published, deployed, or passed cross-lab review. Open items and release blockers are summarized in Document control.

SECTION 12

Appendices

A Glossary

SCP	SolariNet Control Protocol — length-prefixed 32-byte big-endian header + TLV payload + CRC-32, over nng + mbedTLS mTLS.
HRW	Highest-Random-Weight (rendezvous) hashing — coordinator-free assignment of probe targets to monitors.
SoR / CMDB	System of Record / Configuration Management Database — the declared-intent datastore on cesium.
CDC	Change Data Capture — the outbox-and-consume path that propagates SoR changes to DNS/Pi-hole.
solariCtl	The in-process operator bridge inside solariServer; AF_UNIX line protocol; holds CA signing material.
RPZ	Response Policy Zone — the DNS blocklist zone served by BIND on xenon.
F-nn	A numbered finding from the 2026-08-09 architecture review (see §11.4).

B Host quick-index

EVERY ELEMENT NAME → ADDRESS → ONE-LINE ROLE (SEE §5 FOR FULL DETAIL).

HOST	ADDRESS	ROLE	HOST	ADDRESS	ROLE
chemistry	10.0.0.1	gateway (UDR7)	radium	10.1.0.10	AD DC + Keycloak
laboratory	10.0.0.3	core switch	cesium	10.1.0.200	Forgejo + SoR
xenon	10.0.0.20	SolariNet server + DNS	amino	10.1.0.100	n8n
carbon	10.0.0.10	NAS (DXP4800 Plus)	ben-zene	10.5.2.50	MariaDB + net-boot
beaker / flask	10.0.0.7/.8	WiFi 7 APs	nitro-gen	10.5.2.20	dev
helium / mercury	10.0.0.254/.253	Pi-hole pair	quanta	10.6.0.10	Grafana
steel	10.0.0.11	ZimaBlade; tertiary DNS planned after redeployment	va-lence	10.6.0.20	monitor outpost
silicon	10.0.0.250	edge proxy (Caddy)	chlo-rine	10.7.0.10	internal CA + BIND secondary
silver	10.0.0.85	printer	photon	10.7.0.1	NVR hub
hydrogen	10.0.1.50	workstation (Mac)	tachy-on	10.6.6.10	GPS/PTP time

C Methodology & provenance

The host inventory and its discovery observations were captured on 2026-09-17. They were cross-checked against the source tree and live state available on that date. The current code candidate and later live observations are recorded separately in the cover callout and the stabilization baseline. The historic discovery method was `nmap -sn -PR` across the 16 zoned /24s plus read-only SSH fact-gathering; the off-

line IEEE OUI table resolved MAC vendors. A full service/OS scan and SNMP polling were **not** completed — those columns are blank by honest omission, and no MAC, model, or address in this document was synthesized to fill a gap. Where a fact could not be observed, it is marked INFERRED or left as an explicit gap in §5.3.

> **LIVING DOCUMENT**
>
> The inventory is a point-in-time snapshot; hosts move and the SoR drifts. The reconciliation queue in §5.3 (five unnamed hosts, the lithium subnet, the iron/steel rename) is the current delta between the committed record and observed reality. Re-run the discovery pass and this build to refresh.

SolariNet Developer & Architecture Reference · Revision 1.1 · 2026-10-21 · Internal · github.com/soren42/Solarinet · Set in IBM Plex Sans & Mono · Rendered with WeasyPrint from self-contained HTML — no build step.